# Lab-6 Hill Climbing And Local Extrema

To implement the Hill Climbing algorithm for finding a local maximum of the function g

$$g(x) = \sin x + \frac{\sin 3x}{3}$$

and to observe how it effects the final solution.

Step 1: Importing math module

```
import math
```

This imports Python's built in math module. We need it to use the math.sin() function in our objective function.

Variables created: None

Step 2: Define the objective function

```
def g(x):
    return math.sin(x) + math.sin(3*x) / 3
```

This defines the function g(x) that we want to maximise. For a given value of x, the function calculates a score. A higher value of g(x) means a better solution

Variables created: g(x) → objective function

Step 3: Define the function and the variable.

```
def hill_climb(start, step=0.25)
    x = start
```

start is the initial value of x.

step tells the algorithm how far to move to the left or right.

If no step is given 0.25 is used automatically.

Variables created: x - the entire state of algorithm

classmate
Date
Page

Step 4: Open a loop
while True:
This creates a loop that keeps running until the algorithm reaches a position where neither neighbours is better.
The loop runs infinitely until the return statement is executed.
Variables created: x - unchanged

Step 5: Find left and right neighbour
left = x - step
right = x + step
The algorithm checks two positions next to the current position
left is one step left of x
right is one step right of x
Variables created: x, left, right

Step 6: Select the better neighbour
best = left if g(left) > g(right) else right
The function value of the left and right neighbours are compared
best stores the neighbour with higher function value.
Variables created: best - better neighbour, not necessarily better than x

Step 7: Stopping test
if g(best) <= g(x):
    return x, g(x)
The algorithm compares the best neighbour with the current position.
If best neighbour is not better than the current position, the algorithm has reached peak. Therefore it stops and returns the current x and its function value.
Variables created: unchanged

## Step 8: Move to better neighbour

$x$ = best

If a better neighbour exists, the algorithm moves to that position

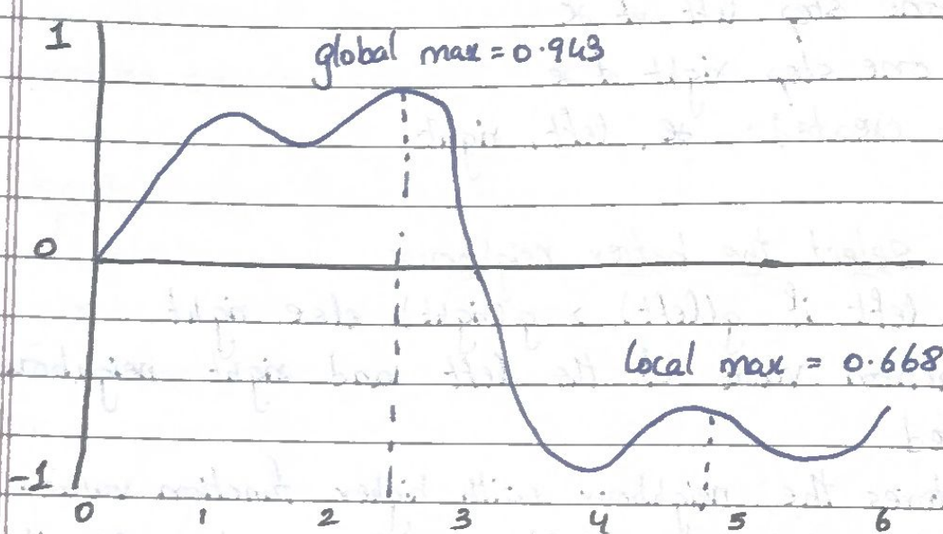
The old value of $x$ is replaced by best and the algorithms goes back to the loop to explore new neighbours

Variables created: $x$ - changed value

## Step 9: Run it twice with different start values:

print ("from $x$ = 0.5", hill_climb(0.5))

print ("from $x$ = 4.0 :", hill_climb(4.0))




For $x = 0.5$

Start: $x = 0.5$

left = $x - 0.25 = 0.25$ $g(0.25) \approx 0.486$

right = $x + 0.25 = 0.75$ $g(0.75) \approx 0.941$

right > left

$\Rightarrow x = 0.75$

left = $x - 0.25 = 0.5$ $g(0.5) \approx 0.86$

right = $x + 0.25 = 1.0$ $g(1.0) \approx 0.655$

$g(0.75) \approx 0.941$ left, right < $x$

$x = 0.75$ $g(x) \approx 0.941$

For $x = 4.0$

Start:

$x = 4.0$

left = $4.0 - 0.25 = 3.75$ $g(3.75) \approx -0.829$

right = $4.0 + 0.25 = 4.25$ $g(4.25) \approx -0.691$

best = 4.25

$x = 4.25$

left = 4 $g(4) \approx -0.706$

right = 4.5 $g(4.5) \approx -0.699$

best = 4.50

$x = 4.50$

left = 4.25 $g(4.25) \approx -0.691$

right = 4.75 $g(4.75) \approx -0.689$

best = 4.75

$x = 4.75$

left = 4.5 $g(4.5) \approx -0.699$

right = 5 $g(5) \approx -0.735$

$g(4.75) \approx -0.668$

best = 4.5

$g(4.5) \leq g(4.75)$ Stop

$x = 4.75$ $g(x) \approx -0.668$